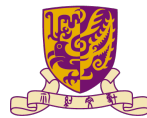


Network Programming Review

Minchen Yu

SDS@CUHK-SZ

Spring 2026



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

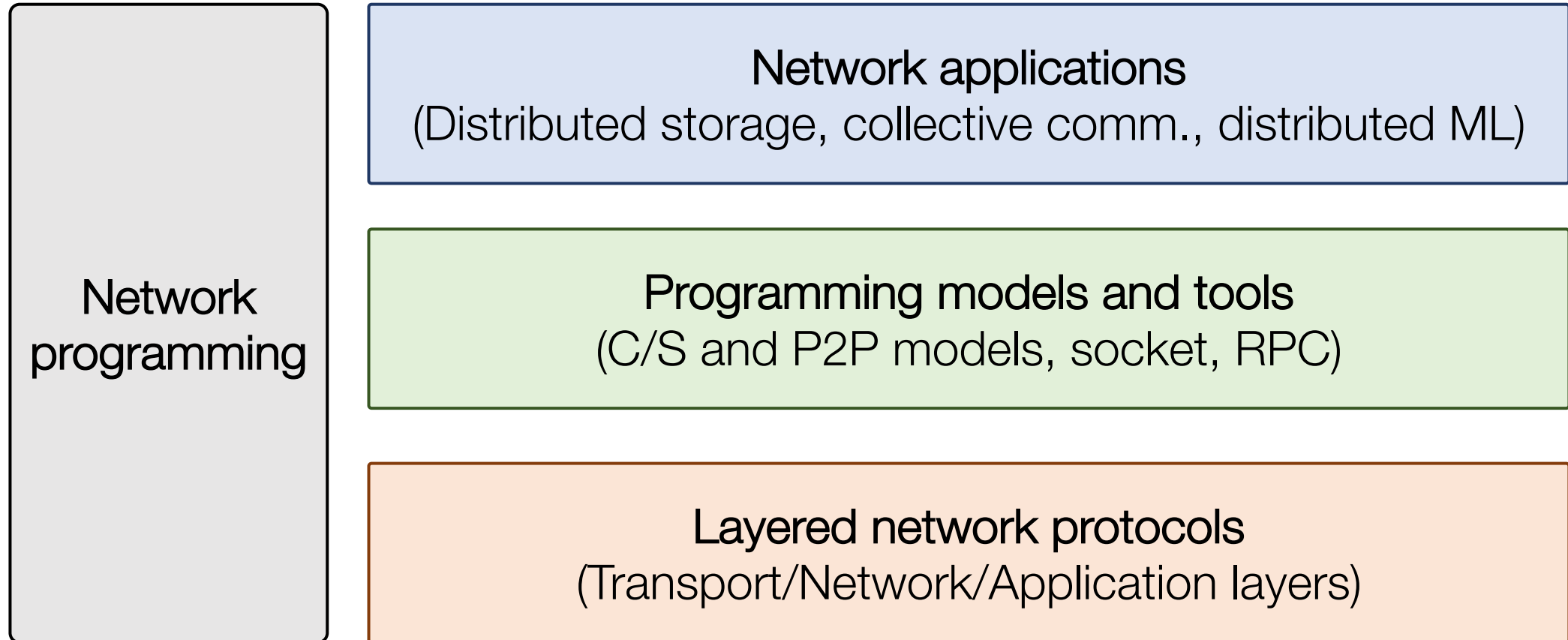


SCHOOL OF
DATA SCIENCE
數據科學學院

Final exam

- Time: 10:30 AM–12:00 PM on May 6
- Venue: The venue will be announced later
- Policy: This is a closed-book exam. Calculators are allowed.

Course overview



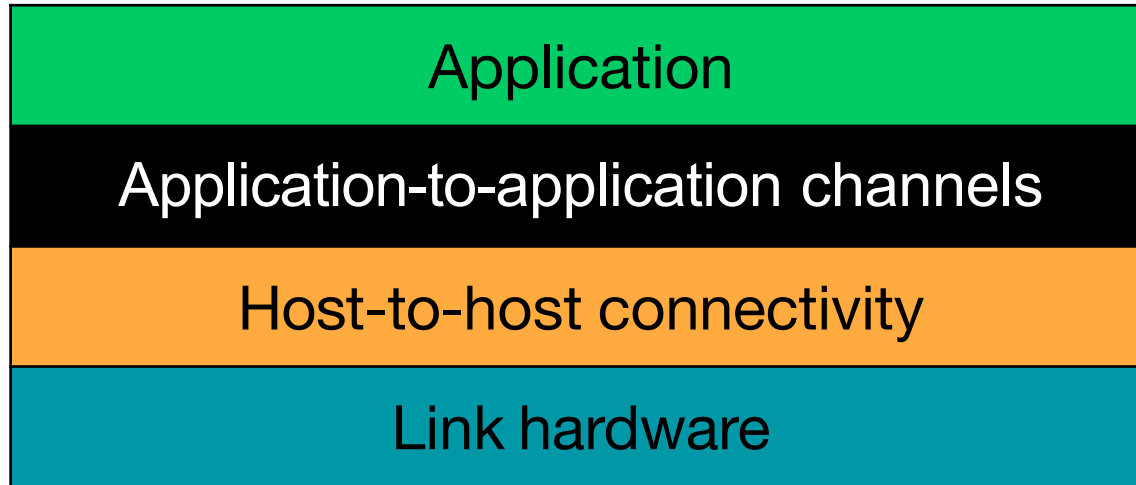
Network protocols

Model of a communication channel

- **Latency:** how long does it take for the first bit to reach destination
- **Capacity:** how many bits/sec can we push through? (often termed “bandwidth”)
- **Jitter:** how much variation in latency?
- **Loss/Reliability:** can the channel drop packets?

What is layering?

- Modular approach to network functionality



Layering characteristics

- Each layer relies on services from layer below and exports services to layer above
- Interface defines interaction with peer on other hosts
- **Protocols** define:
 - Interface to higher layers (API)
 - Interface to peer (syntax & semantics)
 - Actions taken on receipt of a messages
 - Format and order of messages
 - Error handling, termination, etc.
- Hides implementation - layers can change without disturbing other layers (black box)

Reference Model

5	Application	Programs that use network service
4	Transport	Provides end-to-end data delivery
3	Network	Send packets over multiple networks
2	Link	Send frames over one or more links
1	Physical	Send bits using signals

Transport layer protocols

Provide different kinds of data delivery across the network to applications

TCP (Streams)	UDP (Datagrams)
Connections	Datagrams
Bytes are delivered once, reliably, and in order	Messages may be lost, reordered, duplicated
Arbitrary length content	Limited message size
Flow control matches sender to receiver	Can send regardless of receiver state
Congestion control matches sender to network	Can send regardless of network state

UDP and TCP

UDP for apps that don't want reliability or bytestreams, e.g.,

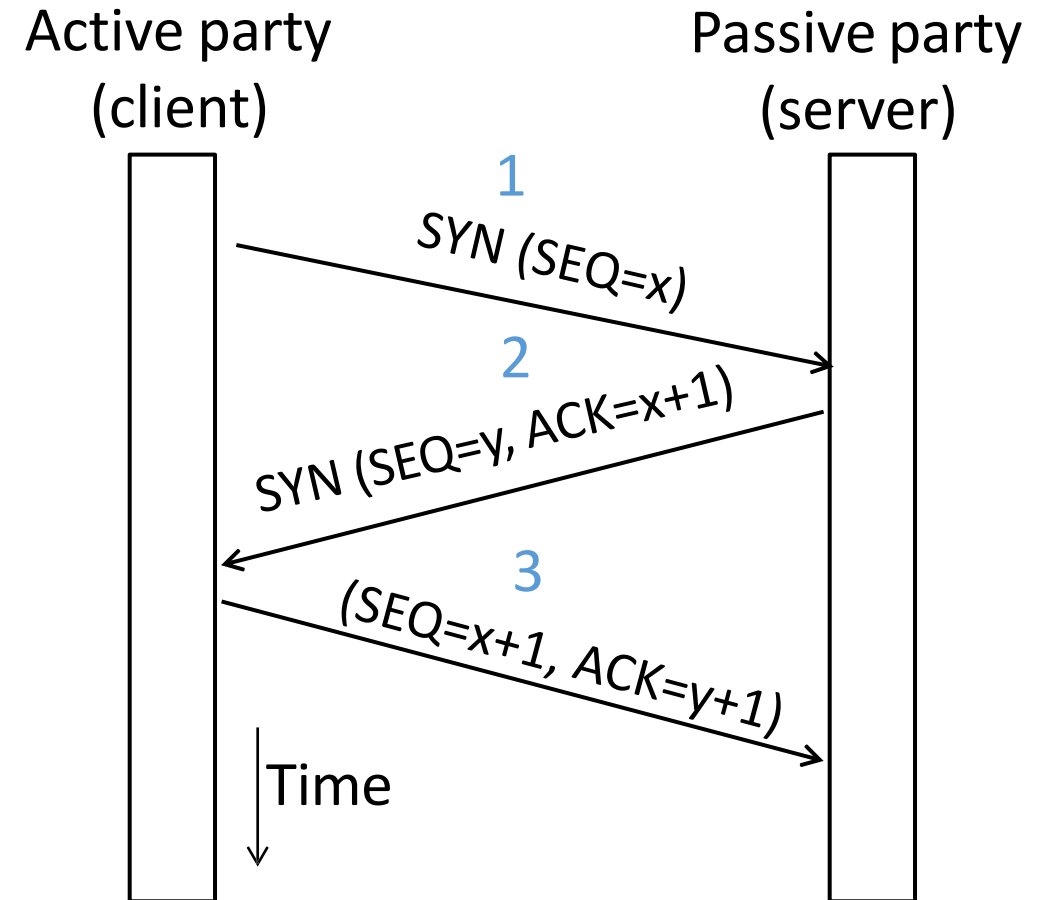
- Voice-over-IP
- DNS
- Games

TCP consists of 3 primary phases:

- Connection Establishment (Setup)
- Sliding Windows/Flow Control
- Connection Release (Teardown)

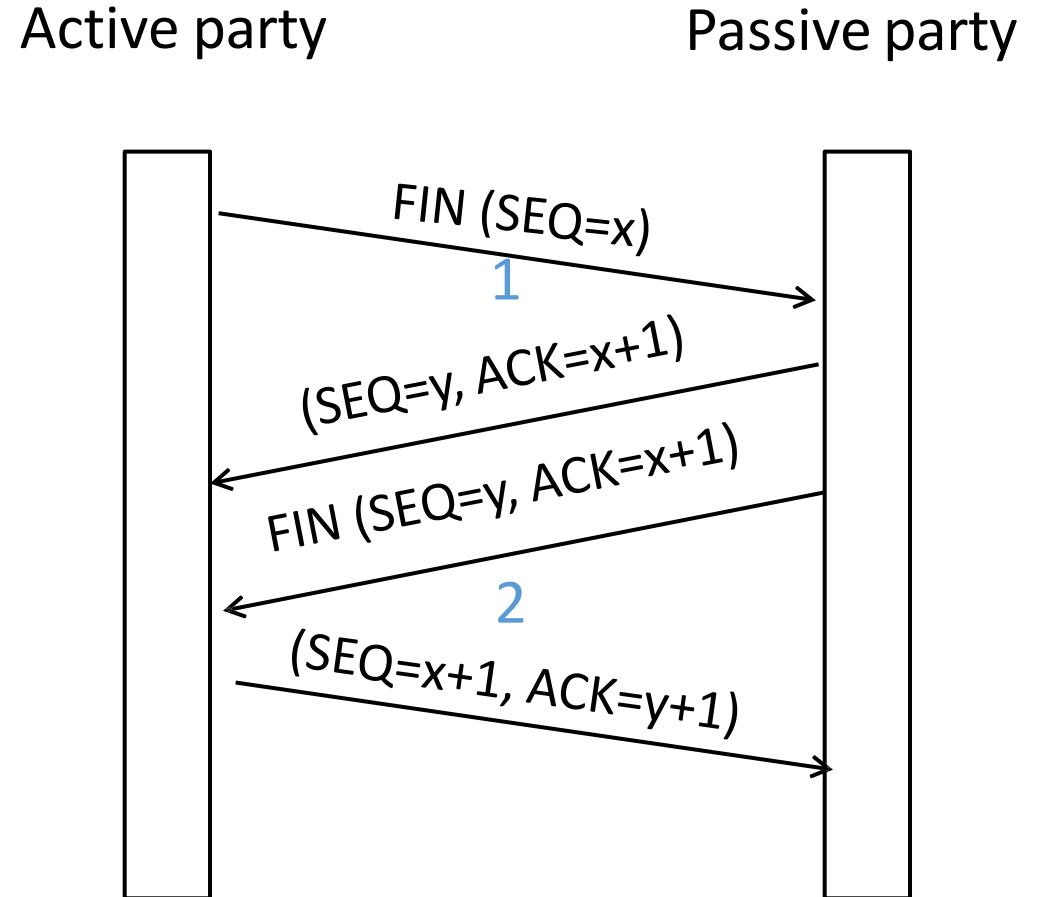
Three-Way Handshake

- Used in TCP; opens connection for data in both directions
- Each side probes the other with a fresh Initial Sequence Number (ISN)
 - Sends on a SYNchronize segment
 - Echo on an ACKnowledge segment
- Chosen to be robust even against delayed duplicates



Connection release

- Two steps:
 - Active sends FIN(x), passive ACKs
 - Passive sends FIN(y), active ACKs
 - FINs are retransmitted if lost
- Each FIN/ACK closes one direction of data transfer

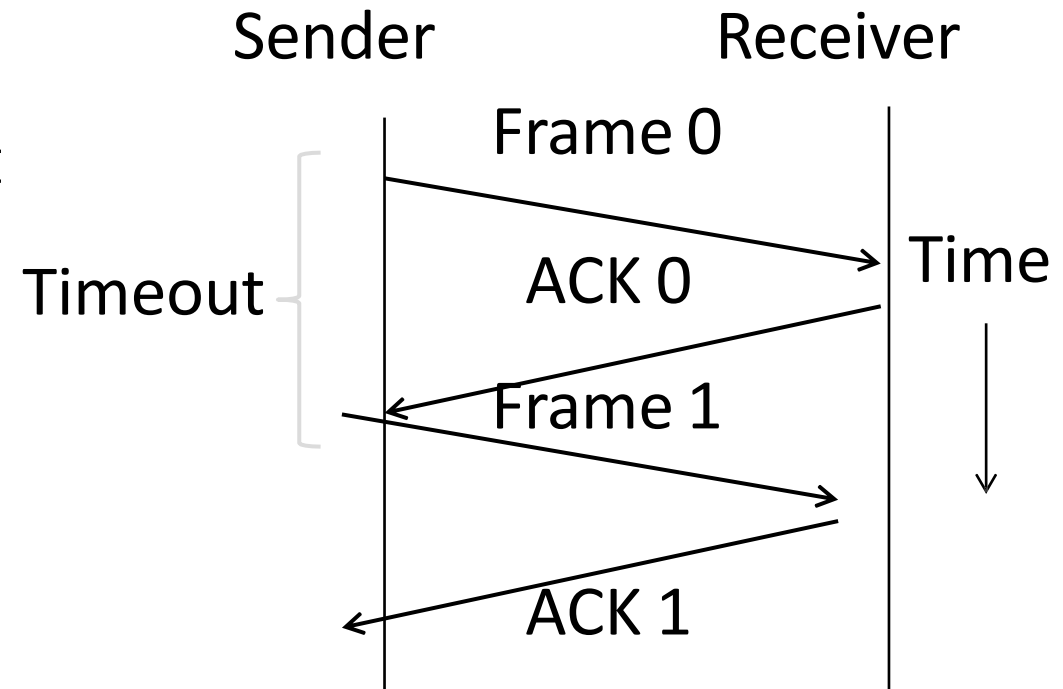


Flow control

Goal: match transmission speed to reception capacity

3 increasingly complex and increasingly efficient solutions

- Stop and wait
- Sliding window: go back N
- Sliding window: selective repeat



Sliding window – Retransmissions

- Go-Back-N uses a single timer to detect losses
 - On timeout, resends buffered packets starting at LAR+1
- Selective Repeat uses a timer per unacked segment to detect losses
 - On timeout for segment, resend it
 - Hope to resend fewer segments

Network congestion

Queues absorb bursts when $\text{input} > \text{output rate}$

But if $\text{input} > \text{output rate}$ persistently, queue will overflow -> congestion

Congestion is a function of the traffic patterns – can occur even if every link has the same capacity

Effects of congestion

- As offered load rises, congestion occurs as queues begin to fill:
 - Delay and loss rise sharply with load
 - Throughput $<$ load (due to loss)
 - Goodput $\ll$ throughput (due to spurious retransmissions)
- None of the above is good!
 - Want network performance just before congestion

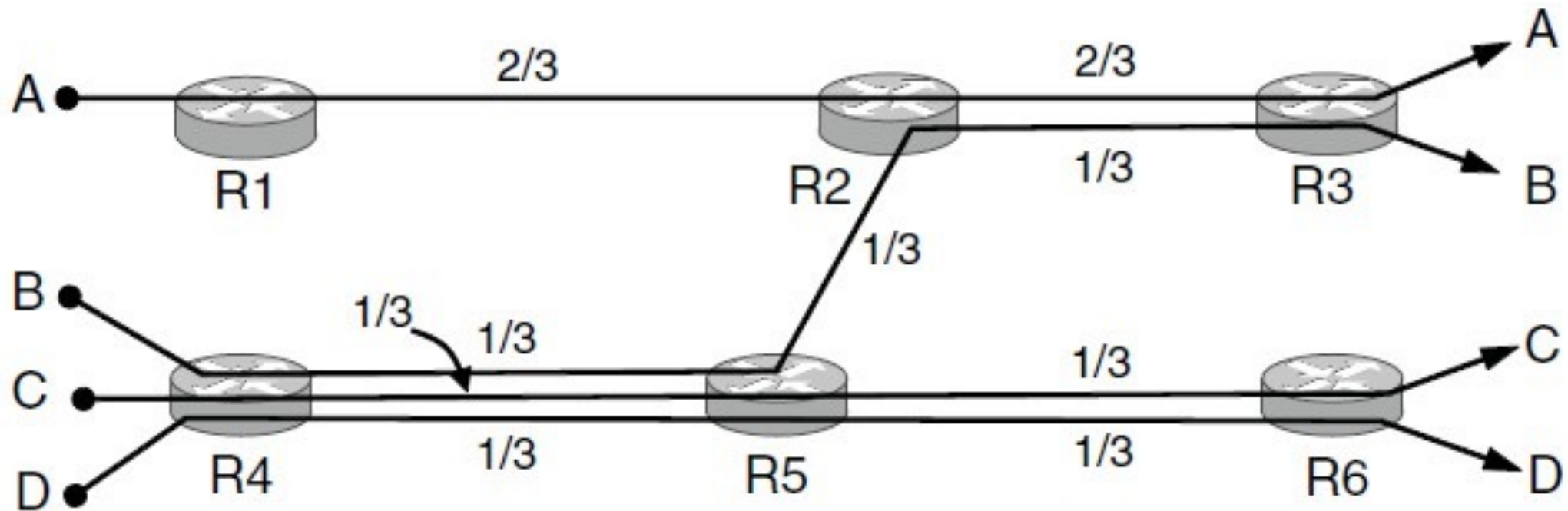


Bandwidth allocation

- Important task for network is to allocate its capacity to senders
 - Good allocation is both efficient and fair
- **Efficient**: most capacity is used but there is no congestion
- **Fair**: every sender gets a reasonable share of the network
- **Max-min fair allocation** is one that:
 - Increasing the rate of one flow will decrease the rate of a smaller flow
 - This “maximizes the minimum” flow

Max-Min example

- End with $A=2/3$, $B, C, D=1/3$, and $R2-R3$, $R4-R5$ full

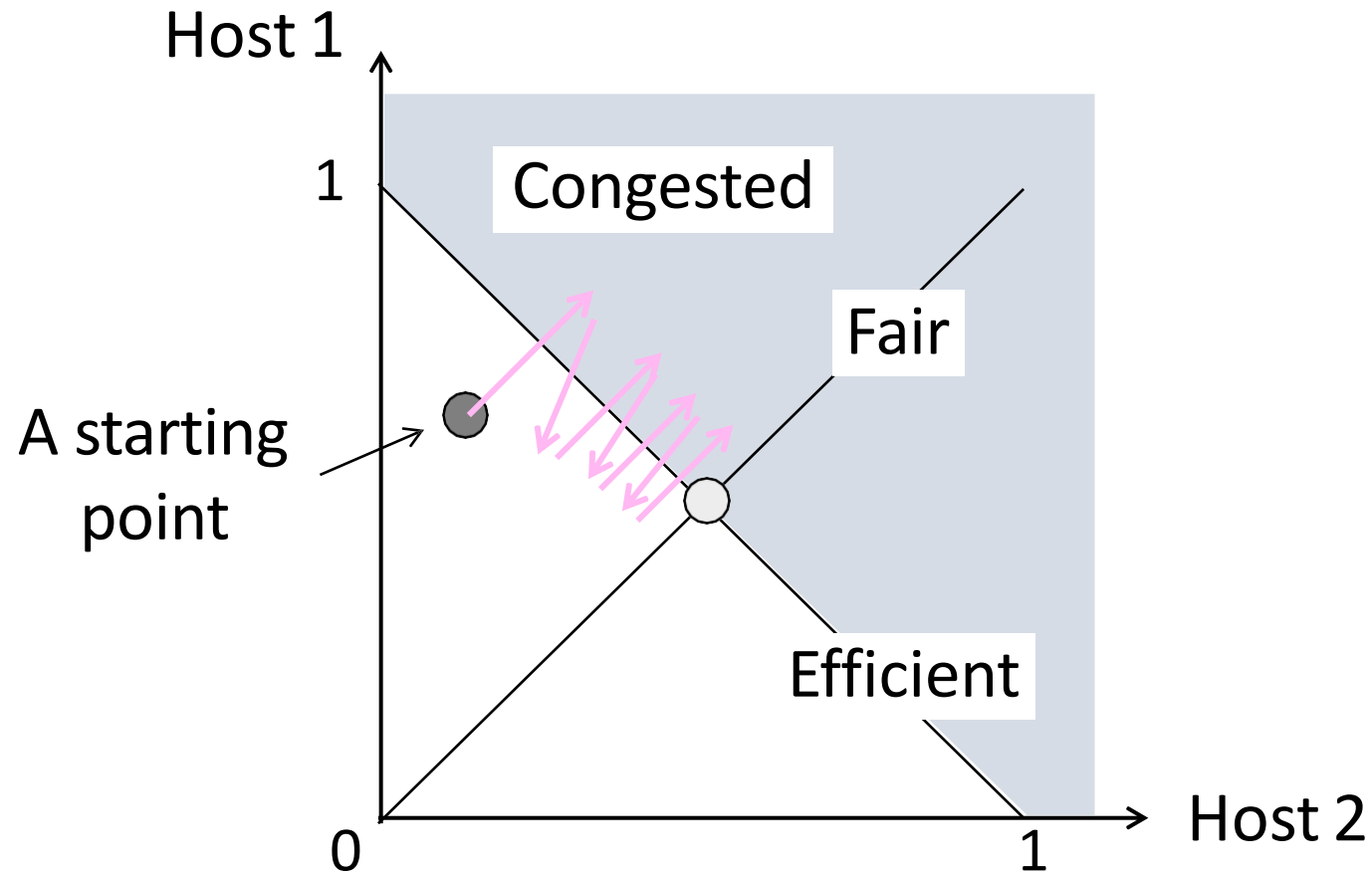


TCP Tahoe/Reno

- TCP extensions and features
 - AIMD
 - Slow-start
 - Fast Retransmission
 - Fast Recovery

AIMD Game

- Always converge to good allocation!



Why do we need a Network (IP) layer?

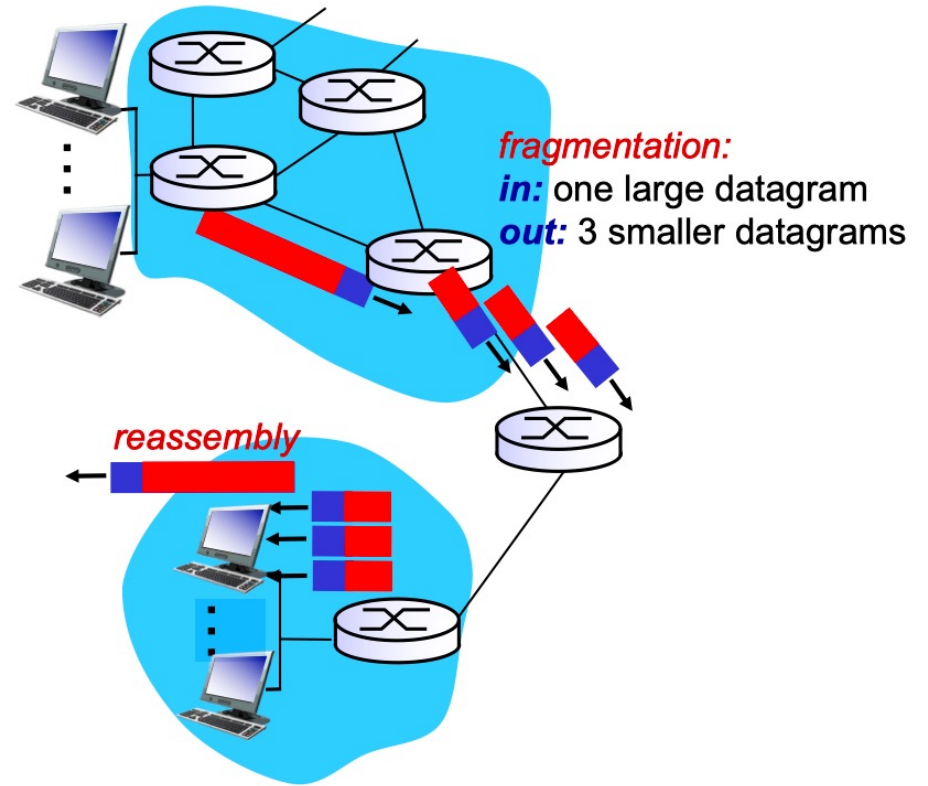
- Internetworking
 - Need to connect different link layer networks
- Addressing
 - Need a globally unique way to “address” hosts
- Routing and forwarding
 - Need to find and traverse paths between hosts

IP Addresses

- IPv4 uses 32-bit addresses
 - IPv6 uses 128-bit addresses
- Addresses are allocated in blocks called prefixes (subnet)
 - Addresses in an L-bit prefix have the same top L bits
 - There are 2^{32-L} addresses aligned on 2^{32-L} boundary
- Written in “IP address/length” notation
 - Address is lowest address in the prefix, length is prefix bits
 - E.g., 128.13.0.0/16 is 128.13.0.0 to 128.13.255.255
 - So a /24 (“slash 24”) is 256 addresses and /32 is 1 address

IP fragmentation

- Network links have MTU (max transfer size)
 - largest possible link-level frame
 - different link types, different MTUs
- Large IP datagram divided (fragmented) within net
 - One datagram becomes several datagrams
 - Reassembled only at final destination

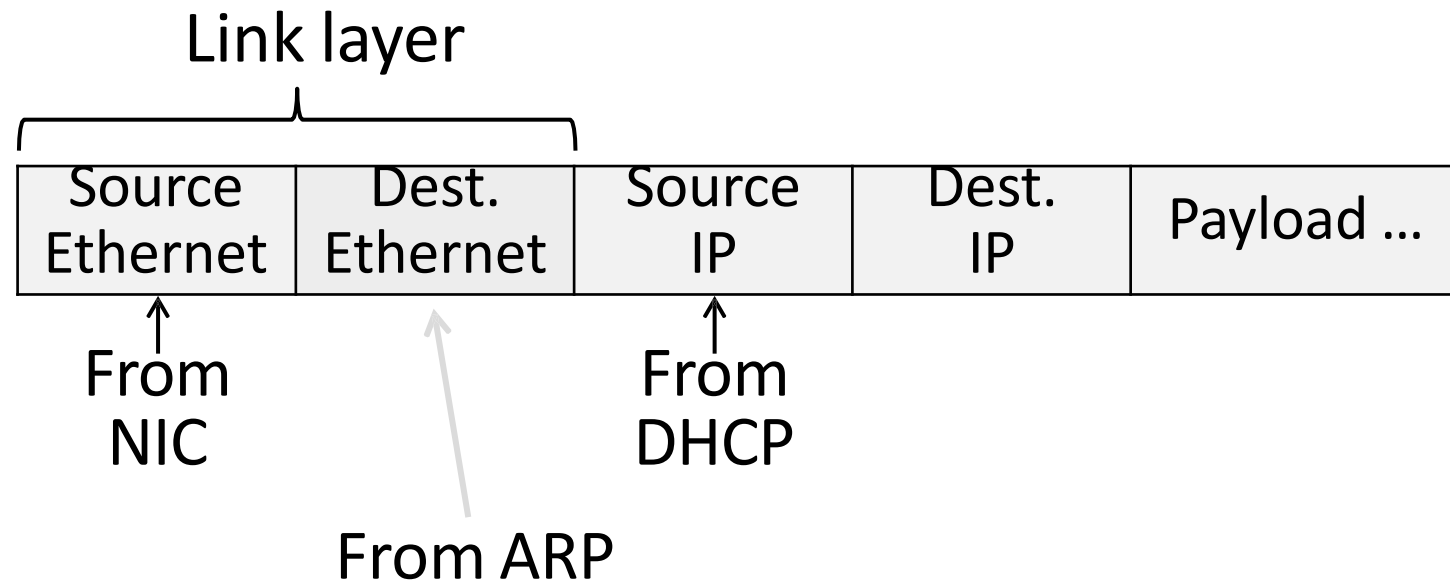


Dynamic Host Configuration Protocol (DHCP)

- Bootstrap issue:
 - How does node send a message to DHCP server before it is configured?
- Answer:
 - Node sends broadcast messages that delivered to all nodes on the link-level network
 - Broadcast address is all 1s
 - IP (32 bit): 255.255.255.255
 - Ethernet (48 bit): ff:ff:ff:ff:ff:ff

Address Resolution Protocol (ARP)

- Node uses to map a local IP address to its Link layer addresses



Internet Control Message Protocol (ICMP)

- ICMP is a companion protocol to IP
 - They are implemented together
 - Sits on top of IP (IP Protocol=1)
- Provides error report and testing
 - Error is at router while forwarding
 - Also testing that hosts can use

Network Address Translation (NAT)

- Map many “Private” IP addresses to one “Public” IP.
- Allocate IPs for private use (192.168.x, 10.x)

Keeps an internal/external translation table

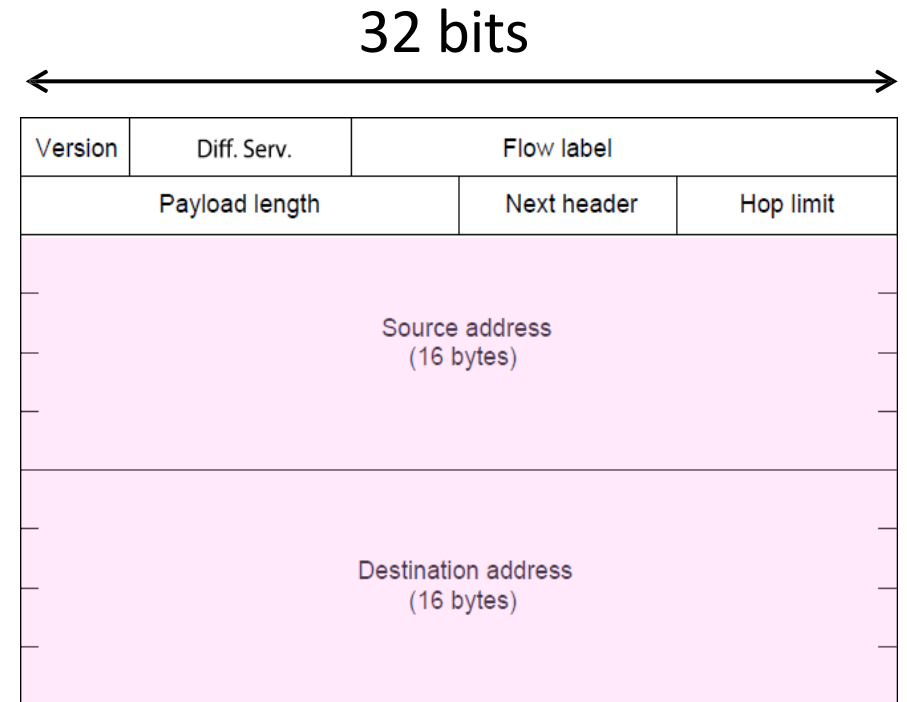
- Typically uses IP address + TCP port
- This is address and port translation

What host thinks	What ISP thinks
Internal IP:port	External IP : port
192.168.1.12 : 5523	44.25.80.3 : 1500
192.168.1.13 : 1234	44.25.80.3 : 1501
192.168.2.20 : 1234	44.25.80.3 : 1502

IP Version 6 (IPv6)

- Features large addresses
 - 128 bits, most of header
- New notation
 - 8 groups of 4 hex digits (16 bits)
 - Omit leading zeros, groups of zeros

Ex: 2001:0db8:0000:0000:0000:ff00:0042:8329
→ 2001:db8::ff00:42:8329



Overview of Routing and Forwarding

- Hosts on same network have IPs in the same IP prefix
- Hosts send off-network traffic to the gateway router
- Routers discover routes to different prefixes (routing)
- Routers use **longest prefix matching** to send packets to the right next hop (forwarding)

Longest Prefix Matching

- Prefixes in the forwarding table can overlap
- Longest prefix matching forwarding rule:
 - For each packet, find the longest prefix that contains the destination address, i.e., the most specific entry
 - Forward the packet to the next hop router for that prefix

Prefix	Next Hop
0.0.0.0/0	A
192.24.0.0/19	B
192.24.12.0/22	C

Two routing algorithms

- Distance vector routing
 - Each node maintains a vector of distances(and next hops) to all destinations and exchange it with neighbors periodically
- Link-state routing
 - Phase 1: flooding
 - Phase 2: each node computes its own forwarding table with Dijkstra's Algorithm

Dijkstra's Algorithm

Algorithm

- Mark all nodes tentative, set distances from source to 0 (zero) for source, and ∞ (infinity) for all other nodes

While tentative nodes remain

- Extract N, a node with lowest distance
- Add link to N to the shortest path tree
- Relax the distances of neighbors of N by lowering any better distance estimates

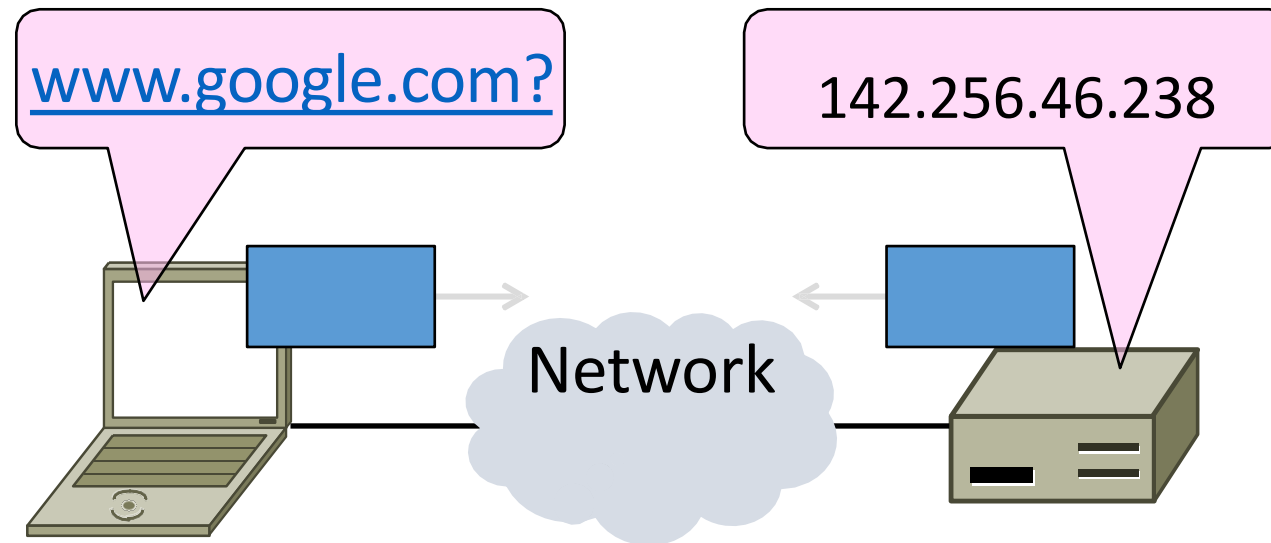
Refer to Lecture 8 for details and examples

HTTP and its performance

- Page load time (PLT) is a key measure of web performance
 - From click until user sees page
 - Small increases in PLT decrease sales
- PLT depends on many factors
 - Structure of page/content
 - HTTP (and TCP!) protocol
 - Network RTT and bandwidth
- Performance optimizations
 - Reduce data size for transfer
 - Parallel connection and connection reuse
 - Web caching and proxies
 - CDNs

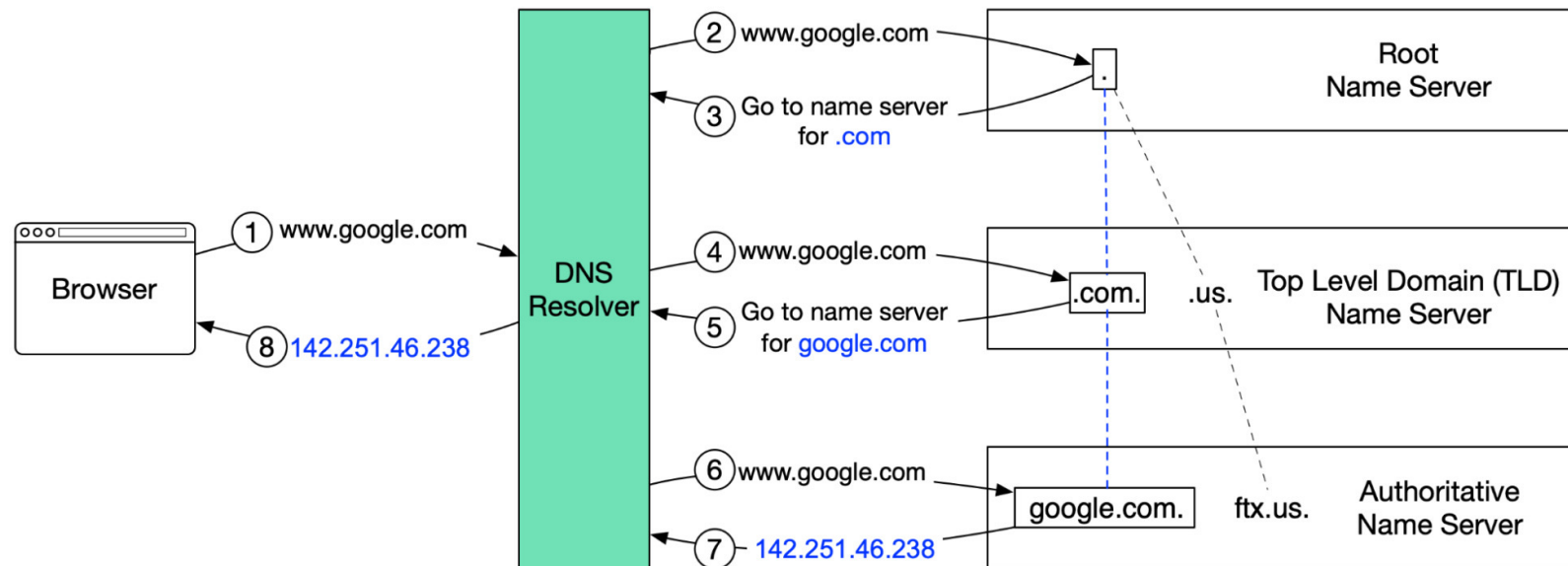
DNS

- Names are higher-level identifiers for resources
- Addresses are lower-level locators for resources
 - Multiple levels, e.g. full name -> email -> IP address -> Ethernet addr
- Resolution (or lookup) is mapping a name to an address



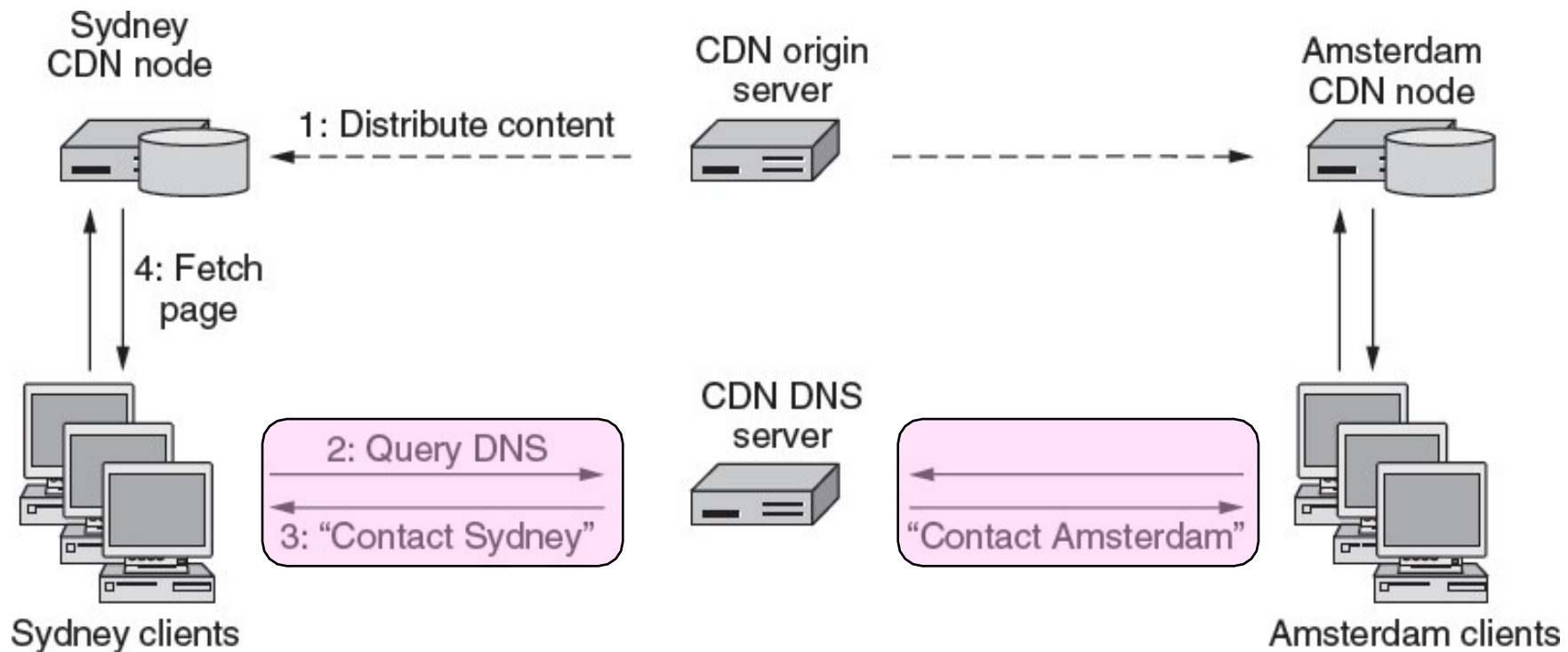
DNS Resolution

- DNS protocol lets a host resolve any host name (domain) to IP address
 - If unknown, can start with the root nameserver and work down zones
- How does DNS resolve IP



DNS and CDN

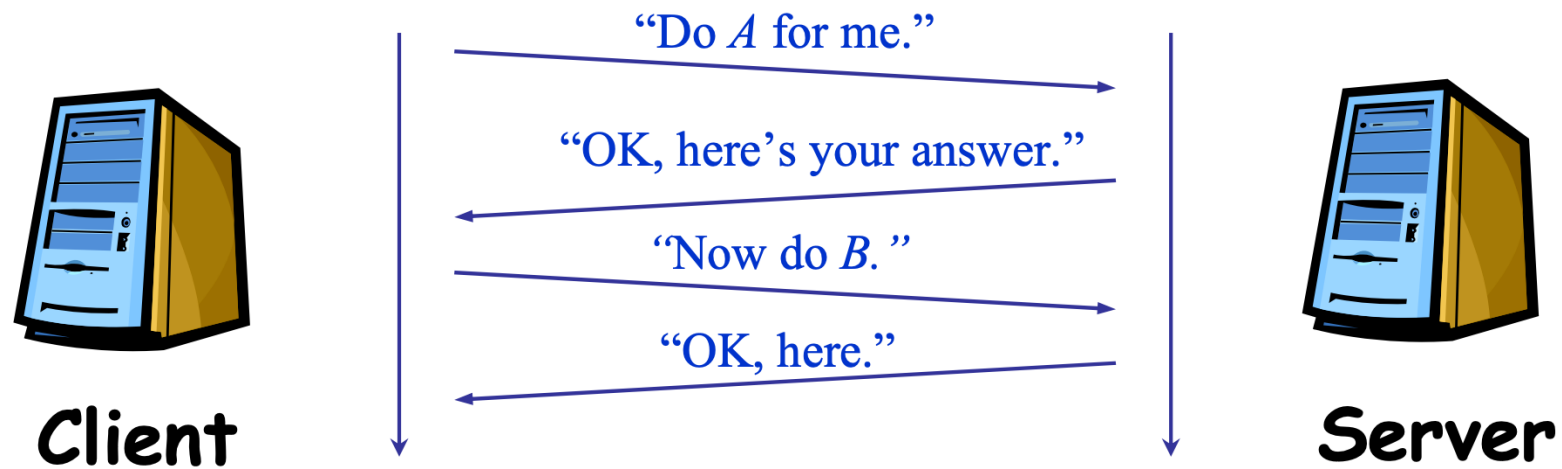
- DNS gives different answers to clients
 - Tell each client the nearest replica (map client IP)



Programming models and tools

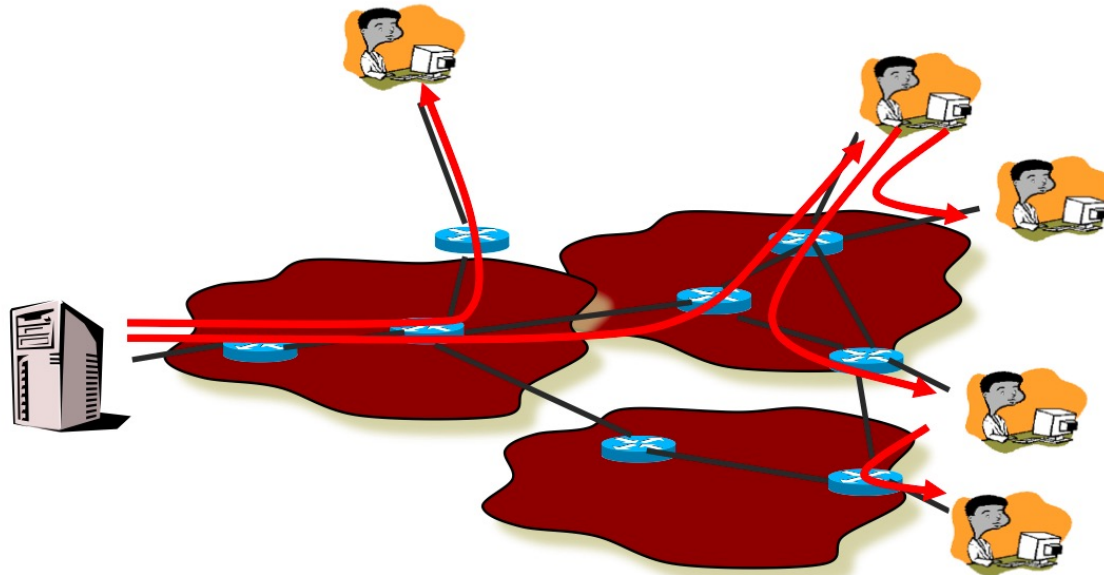
Client/Server model

- Request/response paradigm



P2P model

- Leverage the resources of client machines (peers)
 - Traditional: Computation, storage, bandwidth
 - Non-traditional: mobility, special token we call coins, sensors



P2P frameworks

- Common primitives
 - Join: how do I begin participating?
 - Publish: how do I advertise my file?
 - Search: how to I find a file?
 - Fetch: how to I retrieve a file?
- Napster: centralized database
- Gnutella: decentralized networks and query flooding
- BitTorrent: file swarming and chunk-based downloading

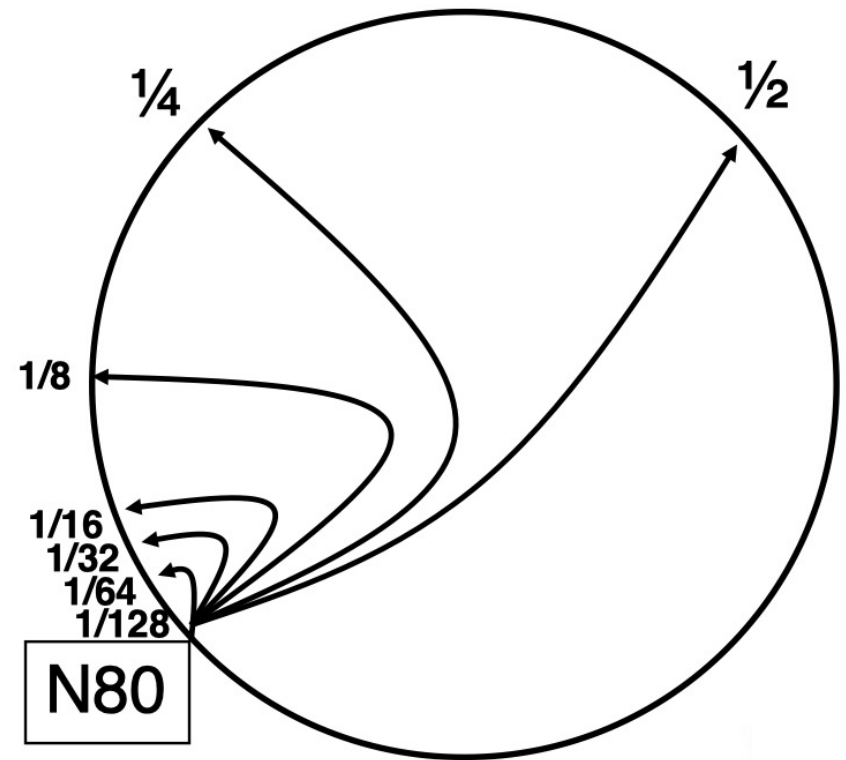
Distributed Hash Table (DHT)

- Goal: make sure that an item (file) identified is always found in a reasonable # of steps
- Abstraction: a distributed hash-table (DHT) data structure

```
key = hash(data)
lookup(key) → IP addr (Chord lookup service)
put(IP address, key, data)
get(IP address, key) → data
```
- Implementation: nodes in system form a distributed data structure
 - Can be Ring, Tree, Hypercube, Skip List, Butterfly Network, ...

Chord finger tables (fast lookup)

- Assume identifier space is $0 \dots 2^m - 1$
- Each node maintains
 - Finger table
 - Entry i in the finger table of n is the first node that succeeds or equals $n + 2^i$
 - Predecessor node
- An item identified by id is stored on the successor node of id



Programming abstraction: socket

- Simple abstraction to use the network
- Set of system-level functions used in conjunction with Unix I/O to build network applications

How to use Socket

- Simple abstraction to use the network
 - Setup socket
 - Where is the remote machine (IP address, hostname)
 - What service gets the data (port)
 - Send and Receive
 - Designed just like any other I/O in UNIX
 - send – write
 - recv – read
 - Close the socket

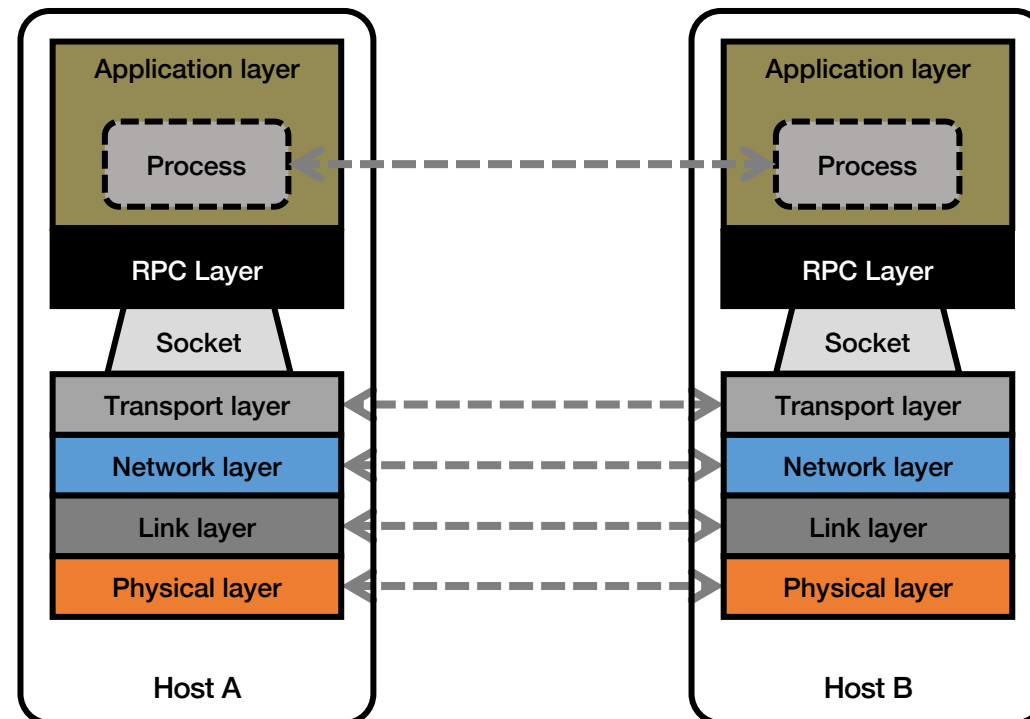
For an application, a socket is a file descriptor

Socket works well enough?

- A lot of programming issues
 - How to separate different requests on the same connection?
 - How to write bytes to the network / read bytes from the network?
What if Host A's process is written in Python and Host B's process is in C++?
 - What to do with those bytes?

RPC

- RPC's Goal: make communication appear like a local procedure call: way less painful than sockets



RPC challenges

- Heterogeneity
 - Client needs to rendezvous with the server
 - Server must dispatch to the required function
 - What if server is different type of machine?
- Failure
 - What if messages get dropped?
 - What if client, server, or network fails?
- Performance
 - Procedure call takes ≈ 10 cycles ≈ 3 ns
 - RPC in a data center takes ≈ 10 μ s ($10^3\times$ slower)
 - In the wide area, typically $10^6\times$ slower

Interface description language

- Mechanism to pass procedure parameters and return values in a machine-independent way
- Programmer may write an **interface description** in the IDL
 - Defines API for procedure calls: names, parameter/return types
- Then runs an **IDL compiler** which generates:
 - Code to **marshal** (convert) native data types into machine-independent byte streams
 - And vice-versa, called **unmarshaling**
 - Client stub: Forwards local procedure call as a request to server
 - Server stub: Dispatches RPC to its implementation

Failure handling

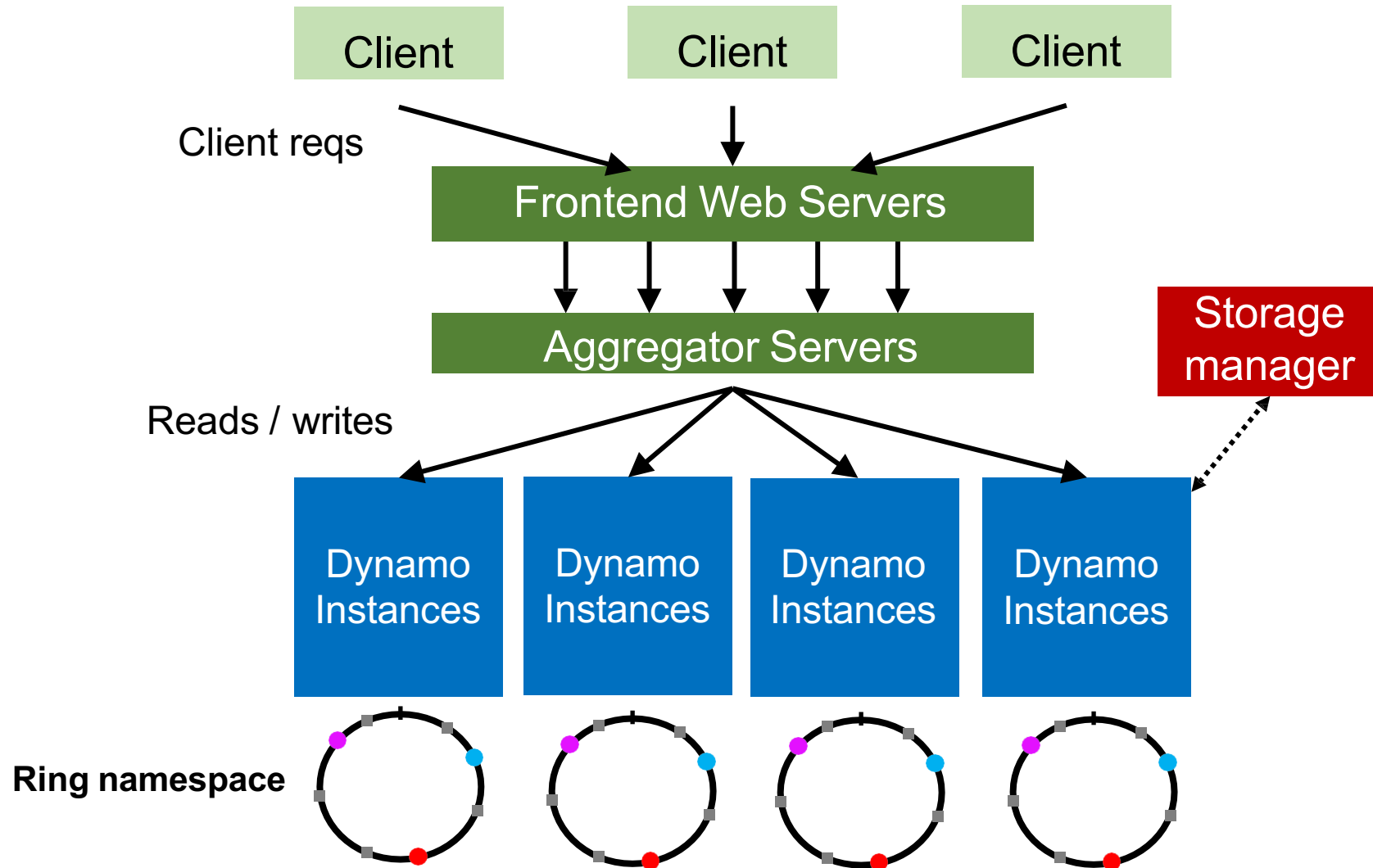
- At-least-once: simplest scheme for handling failures
 - Retransmission
- At-most-once: return repeated reply w/o re-running
 - Duplicate filtering
 - Discard server state w/ cumulative acks
- Exactly-once
 - at-least-once + at-most-once + fault tolerance + no external actions

Network applications

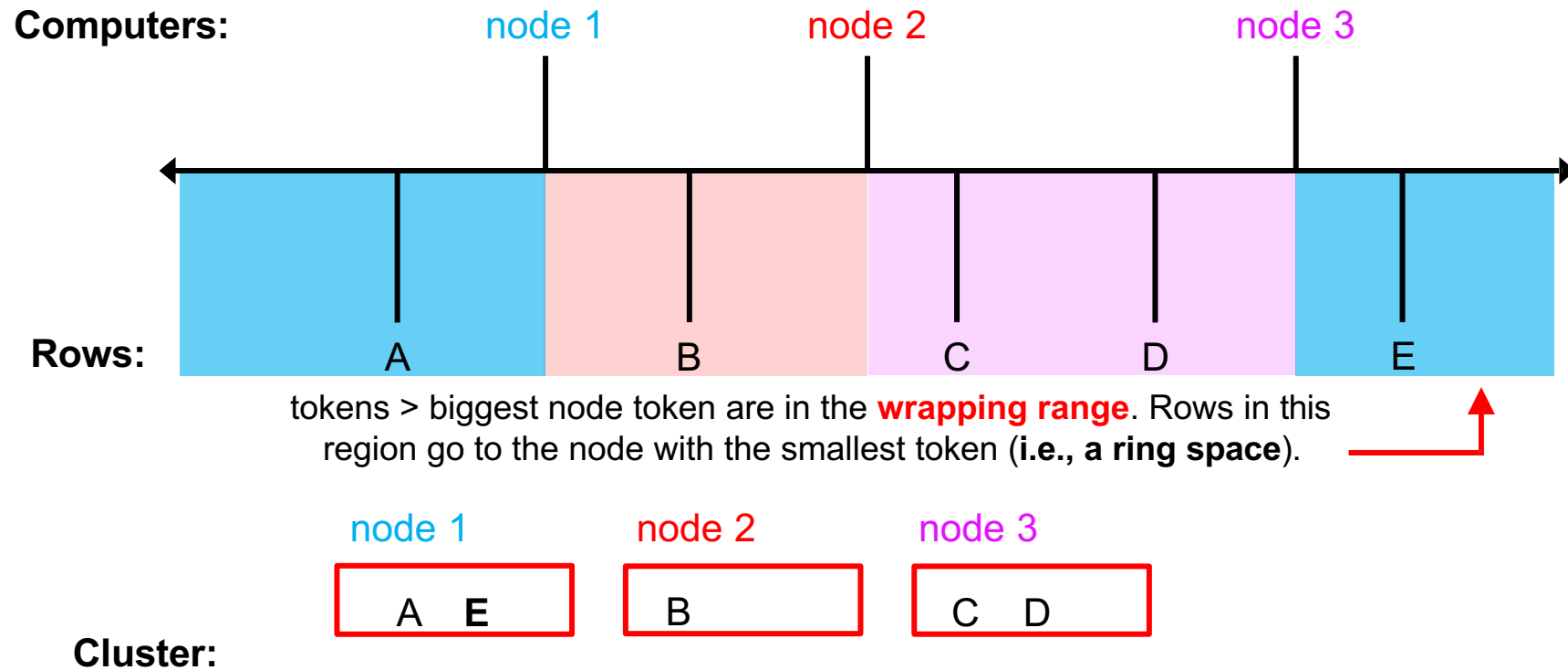
Dynamo scope and requirements

- Scope
 - Shopping cart
 - Others needing a data storage to store lots of states
- Requirements
 - Highly available writes despite failures
 - Always respond quickly, even during failures -> replication
 - Low request-response latency
 - Incrementally scalable as servers grow to workload
 - Adding “nodes” should be seamless
 - Comprehensible conflict resolution
 - High availability in above sense implies conflicts

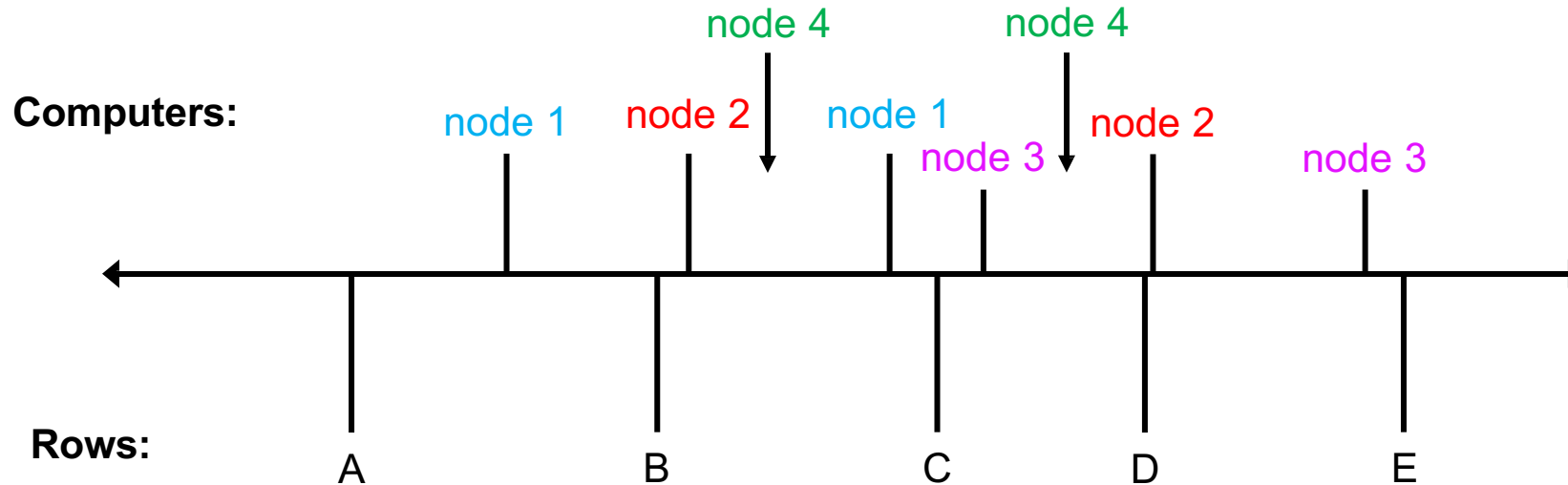
Amazon Dynamo architecture



Consistent hashing



Consistent hashing with virtual nodes



Each (physical) node is responsible for multiple ranges (in this case, 2)

- How many vnodes per node is configurable
- Node 4 will share some load off node 1 and node 2
- Achieves better (short-term and long-term) load balancing with a larger number of vnodes

Replication and read/write quorum

RF	R	W	Behavior
3	2	2	Parameters from the Dynamo paper: Relatively balanced configuration; Good durability, good R/W latency
3	3	1	Slow reads, weak durability, fast writes Writes are highly available, therefore fast; Reads will not return data even if one node is down; reads may fail; Risk: If the one node that took the write fails permanently, we'll lose committed data.
3	1	3	Slow writes, strong durability, fast reads Reads are highly available, therefore fast; Writes are slow (from client's perspective) as they involve writing to three replicas.
3	3	3	More likely that reads see all prior writes
3	1	1	Read quorum doesn't overlap write quorum Speed + availability more important than consistency

GFS/HDFS

A highly reliable storage system built atop
highly unreliable hardwares

Target environment

Thousands of computers

Failures are the norm

Files are huge, but not many

Write-once, read-many

I/O bandwidth is more important than latency

Design decisions

File stores as/divided into chunks

- Large chunk size: 64/128 MB (Why?)

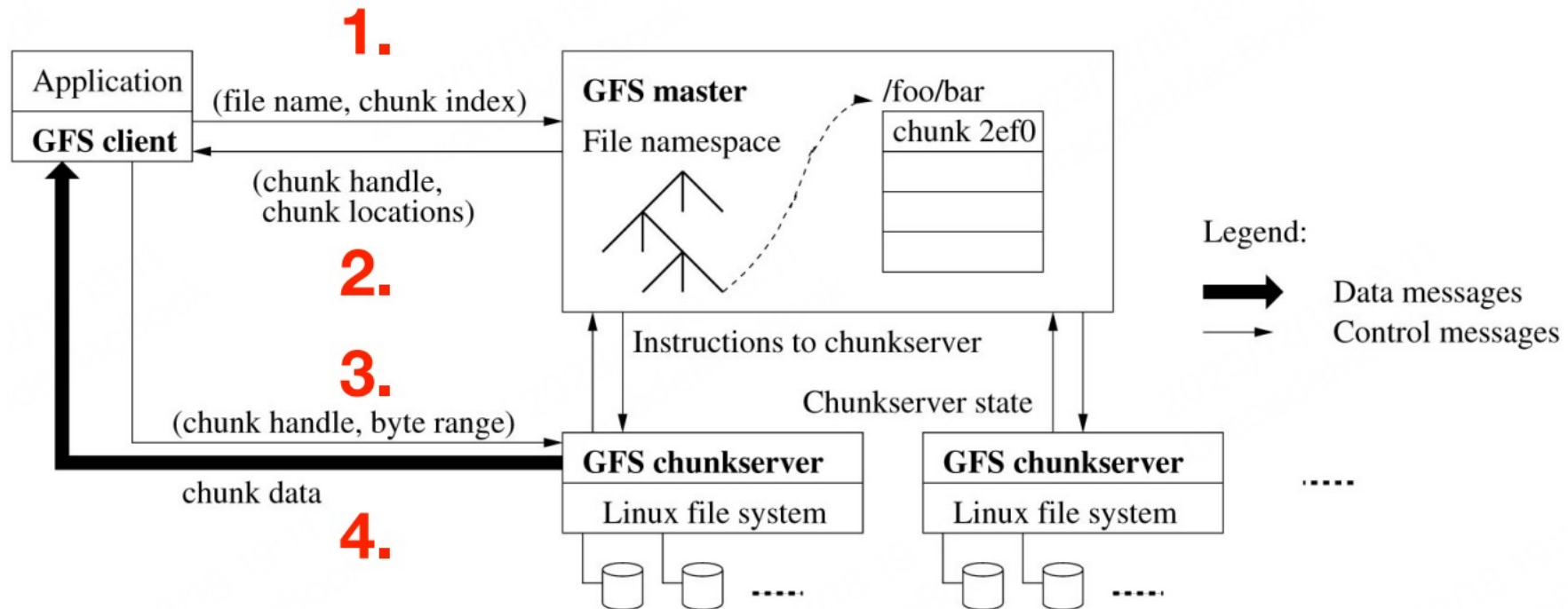
Reliability through replications

- 1 replica in the same rack
- 2+ in other, different racks

Single master to coordinate access

- keep metadata only, file data stored in chunkservers (Why?)

Architecture



Limitations

Assumes write-once, read-many workloads

Assumes a modest number of large files

Emphasizes throughput over latency

GFS/HDFS vs. Dynamo

Aspect	GFS /HDFS	Dynamo
Interface	- File system	- Key-value store
Assumptions	- Large files - write-once, (Sequential) read-many	- Small object storage - Random write & read
Primary Targets	- Batch processing - High throughput - Fault tolerance at scale	- Transactional workloads - Data availability & low latency response - Conflict resolution
Design Choices	- Master-worker architecture - Cross-rack replication - Relaxed consistency	- Peer-to-peer architecture - Consistent hashing - Eventual consistency

Time synchronization

- “Wall clock time” synchronization
- Logical time synchronization (Lamport clocks)
- Vector clock

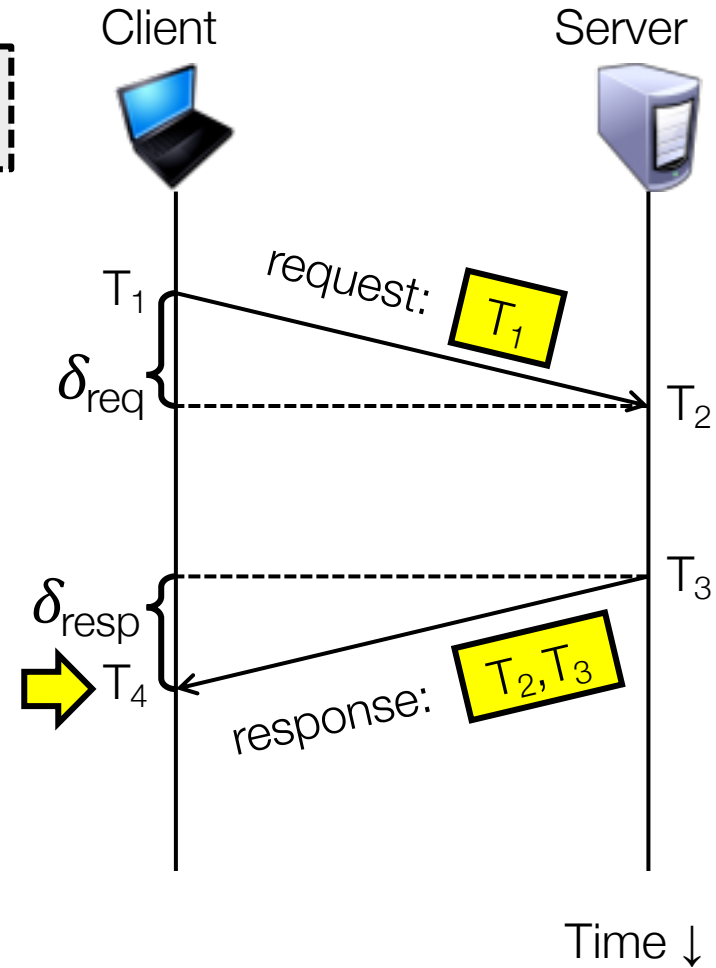
Cristian's algorithm: Offset sample calculation

Goal: Client sets clock $\leftarrow T_3 + \delta_{\text{resp}}$

- Client samples round trip time (δ)
 $\delta = \delta_{\text{req}} + \delta_{\text{resp}} = (T_4 - T_1) - (T_3 - T_2)$
- But client knows δ , not δ_{resp}

Assume: $\delta_{\text{req}} \approx \delta_{\text{resp}}$

Client sets clock $\leftarrow T_3 + \frac{1}{2}\delta$



Lamport Timestamps: Ordering all events

- Break ties by appending the process number to each event:
 1. Process P_i timestamps event e with $C_i(e).i$
 2. $C(a).i < C(b).j$ when:
 - $C(a) < C(b)$, or $C(a) = C(b)$ and $i < j$
- Now, for any two events a and b , $C(a) < C(b)$ or $C(b) < C(a)$
 - This is called a total ordering of events

Vector clock: Introduction

- One integer can't order events in more than one process
- So, a **Vector Clock (VC)** is a vector of integers, one entry for each process in the entire distributed system
 - Label event e with $VC(e) = [c_1, c_2, \dots, c_n]$
 - Each entry c_k is a count of events in process k that causally precede e

Parallelism

$$\textbf{Speedup} = \frac{\text{Completion time given 1 worker}}{\text{Completion time given } N \text{ worker}}$$

Speedup affected by:

- The “parallelizable” portion of a program
- Degree of parallelism
- Execution times of individual tasks
- Synchronization and communication

...

Communication patterns

Point-to-point communication

- Single sender, single receiver
- Relatively easy to implement efficiently

Collective communication

- Multiple senders and/or receivers
- Complex patterns (broadcast, scatter, gather, ...)

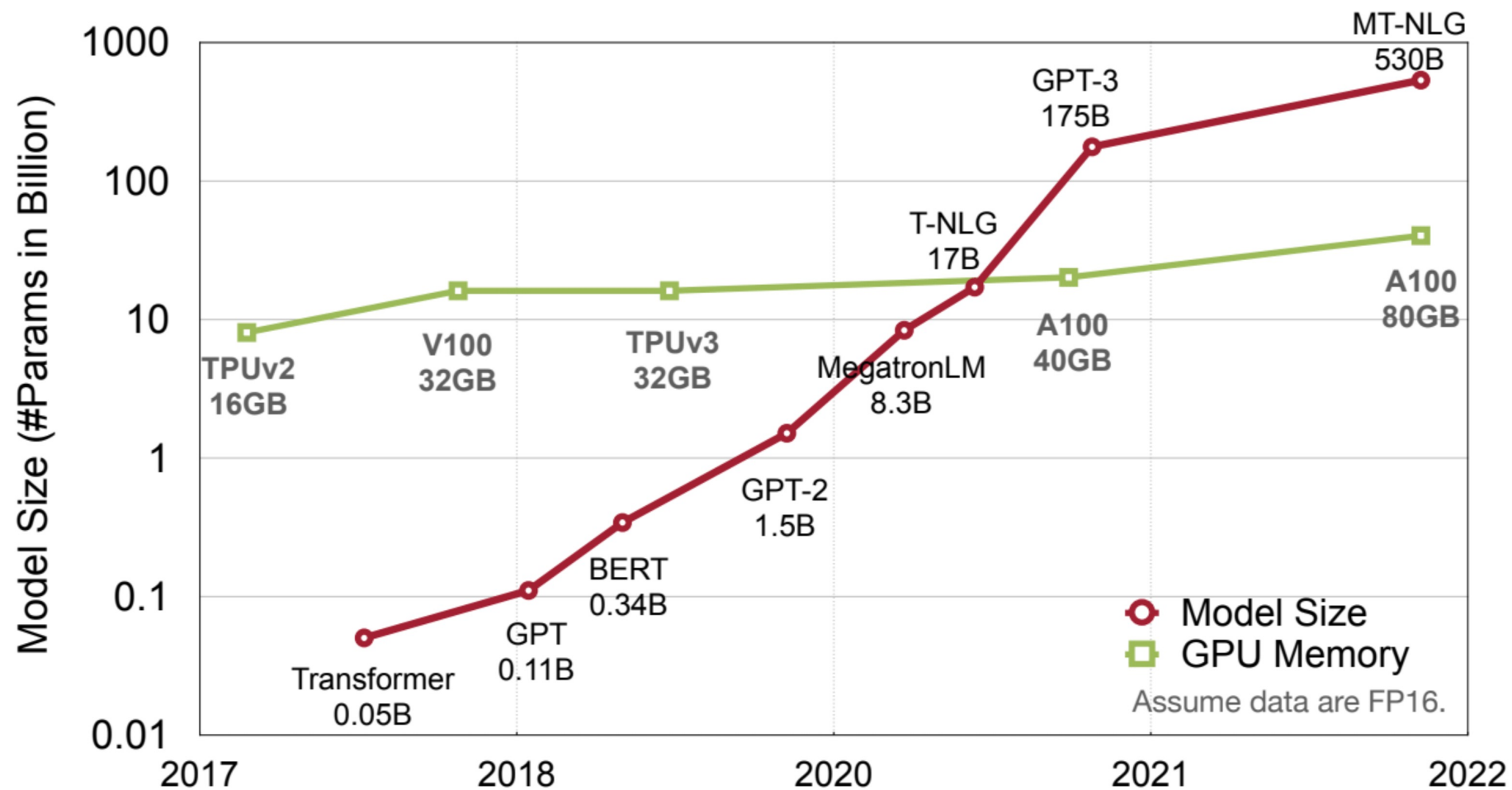
Collective communication primitives

- Broadcast
- Reduce(-to-one)
- Scatter
- Gather
- Allgather
- Reduce-scatter
- Allreduce

Two Family of Mainstream Algorithms

- Small message: Minimum Spanning Tree algorithm
 - Emphasize **low latency**
- Large Message: Ring algorithm
 - Emphasize **bandwidth utilization**

Need of distributed ML



71

LLM serving and continuous batching

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3					
S_4	S_4	S_4	S_4				

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Benefits:

- Higher GPU utilization
- New requests can start immediately